PRACTICAL 5

Aim

To perform Create, Read, Update, and Delete (CRUD) operations using MongoDB and understand how a NoSQL document database stores, manages, queries, and analyzes semistructured data using MongoDB Compass.

Problem Definition

A college wants to maintain student information using a flexible NoSQL database. The database should store student records such as student ID, name, age, course, semester, city, marks, and skills.

As a database administrator, the responsibility is to create and manage a MongoDB database using MongoDB Compass, perform CRUD operations, apply filters, create indexes, analyze query performance, and use the MongoDB aggregation pipeline for data analysis.

The practical includes creating a database and collection, inserting single and multiple documents, reading and filtering documents, updating existing documents, deleting documents, creating indexes, importing JSON data, and performing aggregation operations.

Objectives

- Understand MongoDB and NoSQL databases.
- Create a MongoDB database and collection using MongoDB Compass.
- Insert single and multiple documents.
- Read and display documents from a collection.
- Perform filtering and query operations.
- Update existing documents.
- Delete documents and records.
- Create indexes on MongoDB collections.
- Compare query performance using query analysis.
- Understand BSON document storage.
- Perform aggregation using the MongoDB aggregation pipeline.
- Use $match, $group, $sort, and $limit stages.
- Import JSON data into MongoDB.
- Perform analytical queries on MongoDB data.
- Understand document-oriented data management.

Procedure

1. Open **MongoDB Compass**.

2. Connect to the local MongoDB server.
3. Create a database named collegeDB.
4. Create a collection named students.
5. Insert a single student document.
6. Insert multiple student documents.
7. Display all documents from the collection.
8. Apply filters to search specific student records.
9. Perform queries using comparison operators.
10. Update an existing student document.
11. Delete an unnecessary document.
12. Create an index on student_id.
13. Analyze query performance using the Explain Plan.
14. Open the Aggregations tab.
15. Perform aggregation using $match, $group, $sort, and $limit.
16. Calculate average marks.
17. Count students according to course and city.
18. Find the top students according to marks.
19. Create a JSON dataset.
20. Import the JSON dataset into MongoDB Compass.
21. Perform analytical queries on the imported data.
22. Verify the results.
23. Observe the changes in the collection.

Key Questions / Analysis

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is a NoSQL document-oriented database that stores data in flexible BSON documents. Unlike relational databases, which organize data into tables, rows, and columns, MongoDB stores data in collections and documents. MongoDB provides flexible schemas and is suitable for semi-structured and changing data.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update, and Delete. These are the basic operations required to manage data in a database. • Create – Insert new data.

- Read – Retrieve existing data.
- Update – Modify existing data.
- Delete – Remove data.

CRUD operations are fundamental because they provide the basic mechanisms required to manage database records.

Q3. How does MongoDB store data in BSON format?

MongoDB stores documents internally using BSON, which stands for Binary JSON. BSON is a binary representation of JSON-like documents and supports additional data types such as dates, binary data, and ObjectId.

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline is a framework used to process and analyze multiple MongoDB documents through a sequence of stages. Each stage performs a specific operation such as filtering, grouping, sorting, or limiting documents.

Common stages include:

$match

$group

$sort

$limit

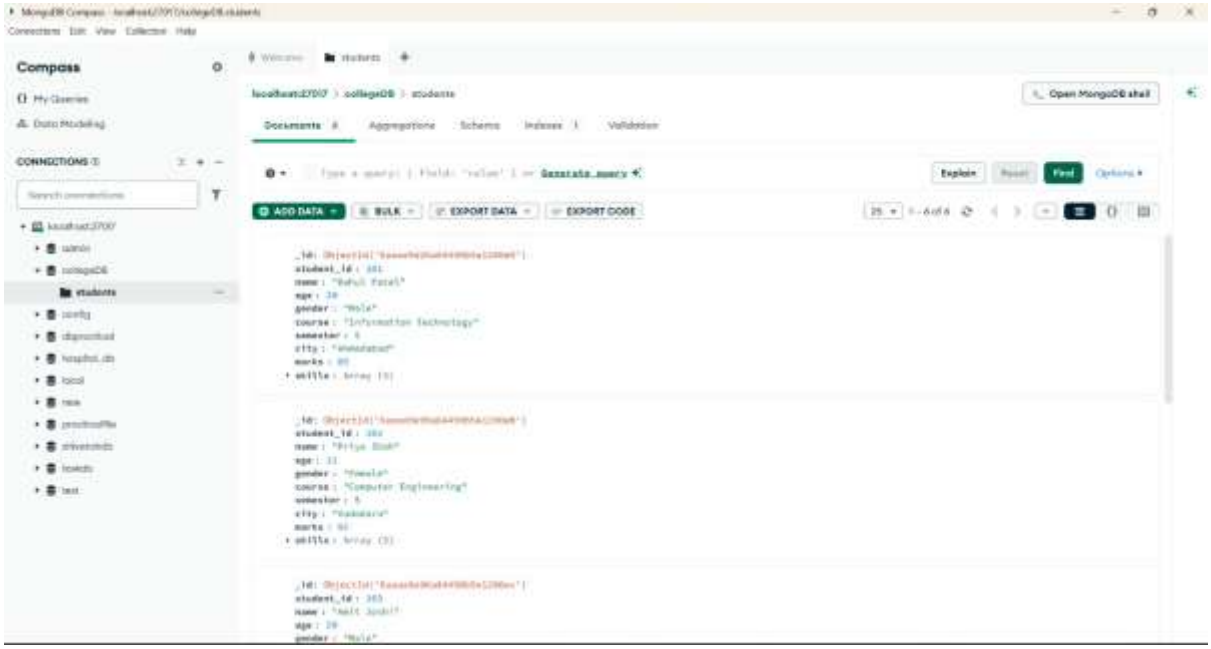
Q5. When should NoSQL databases be preferred over SQL databases?

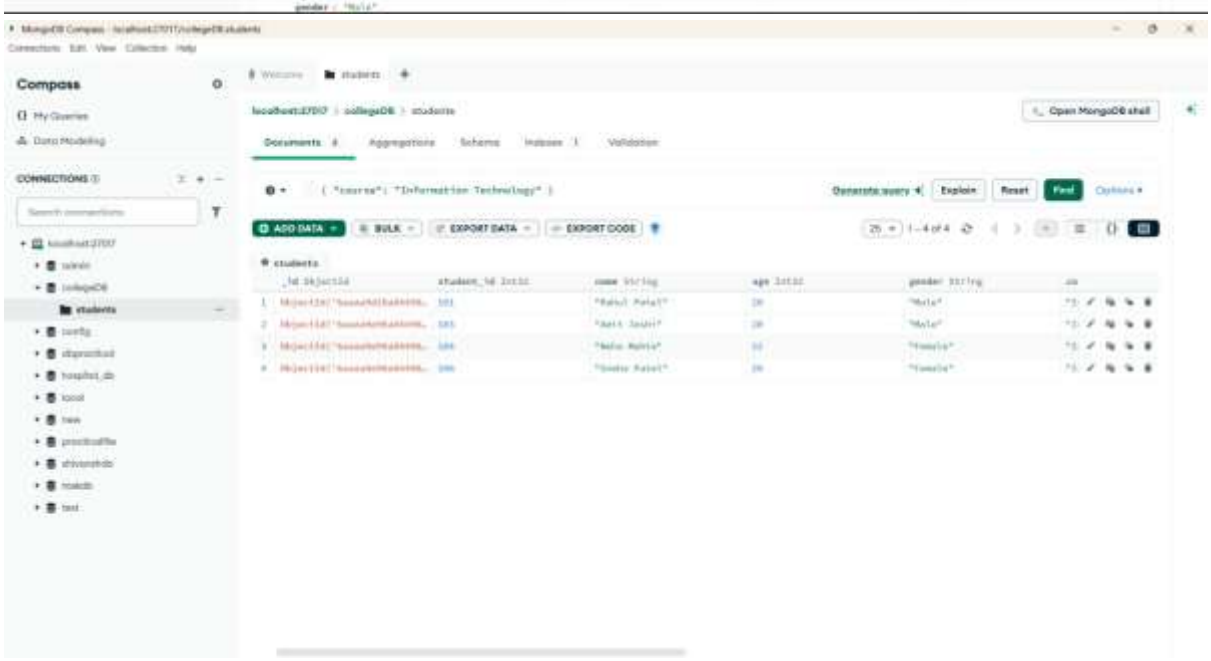
NoSQL databases can be preferred when applications require flexible schemas, semistructured or unstructured data, high scalability, and large volumes of rapidly changing data. SQL databases are generally suitable when structured relational data and strong relationships between tables are central to the application.

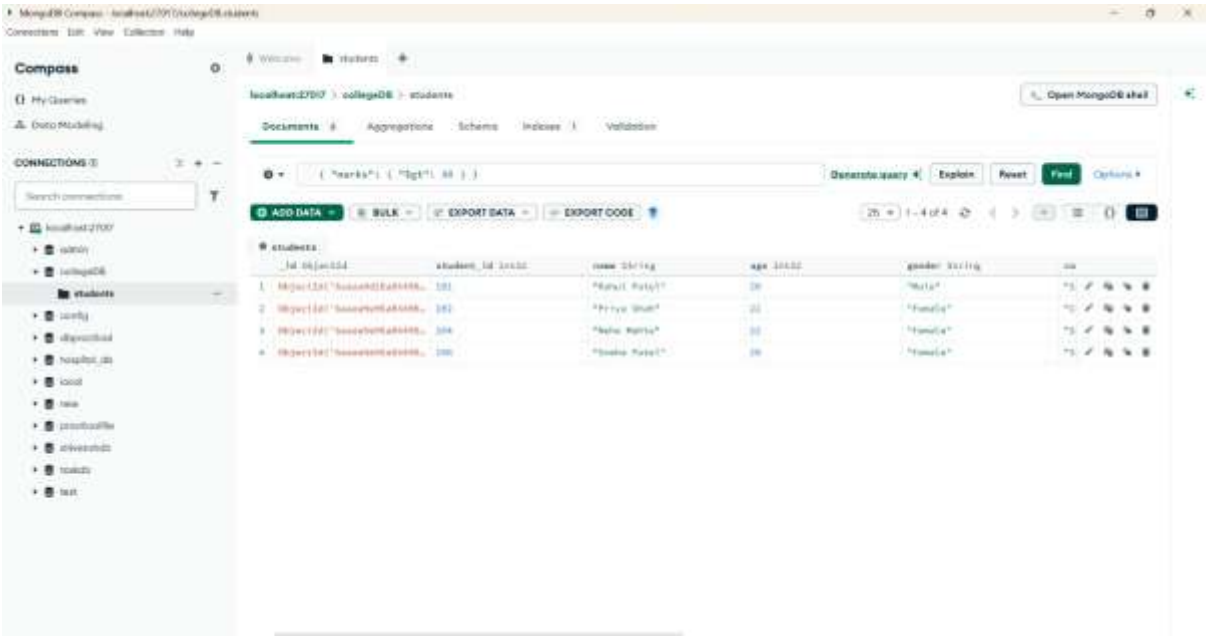
Supplementary Problems

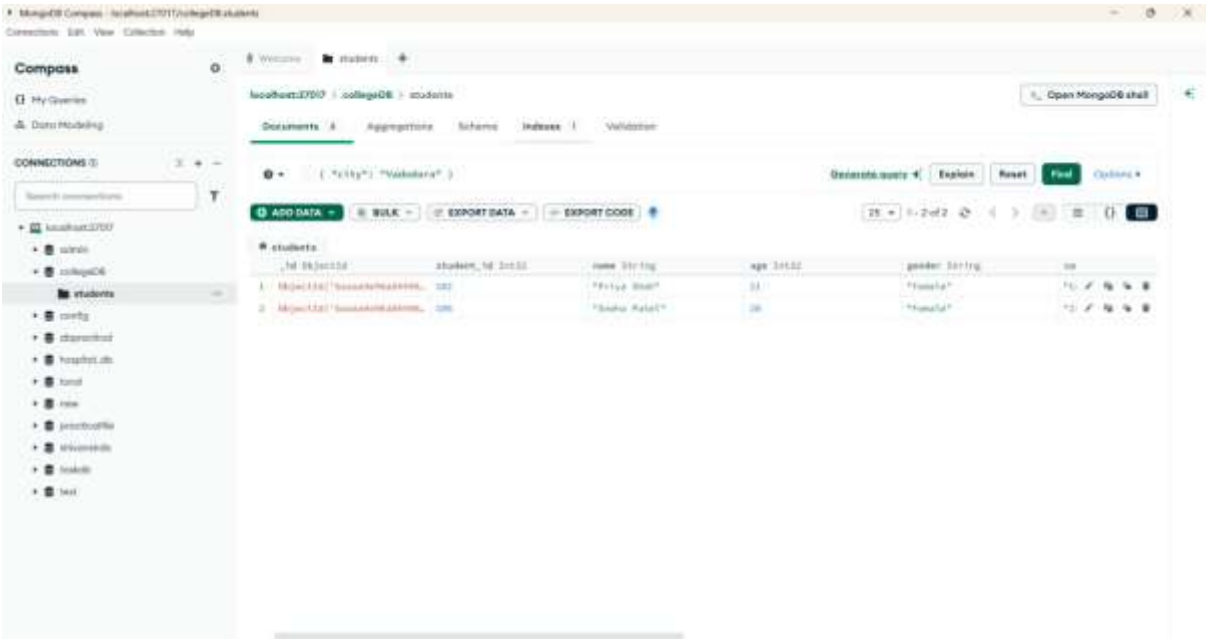
1. Create an index on different fields and compare query performance.
2. Import a large JSON dataset into MongoDB.
3. Create an aggregation pipeline to calculate statistical summaries.

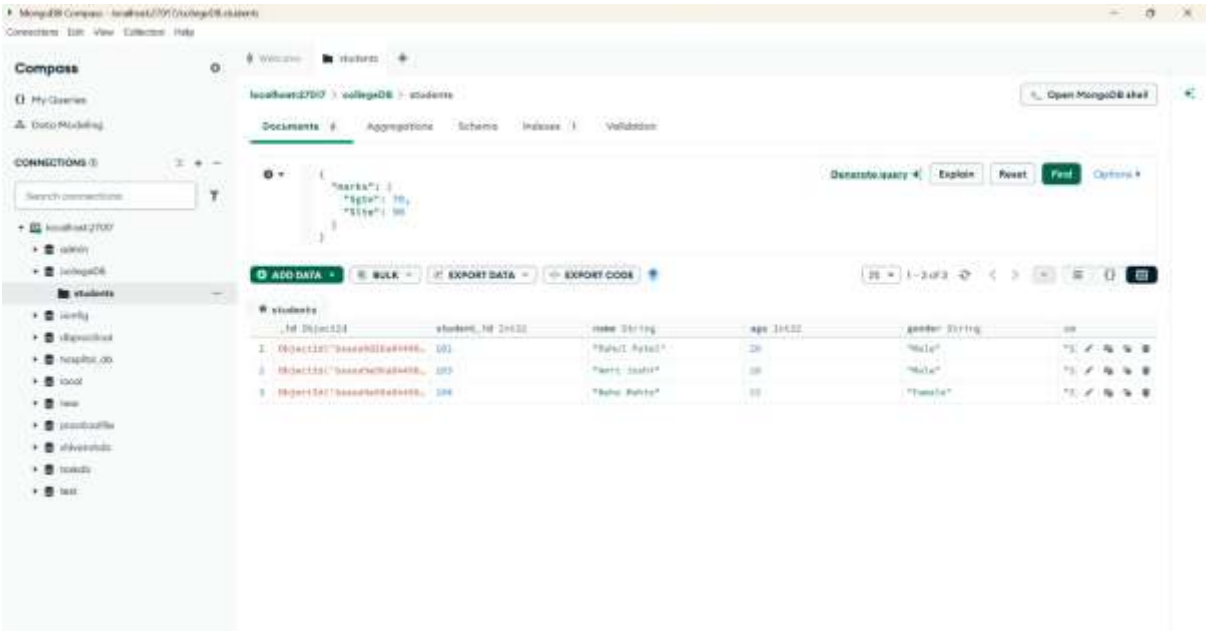
Output

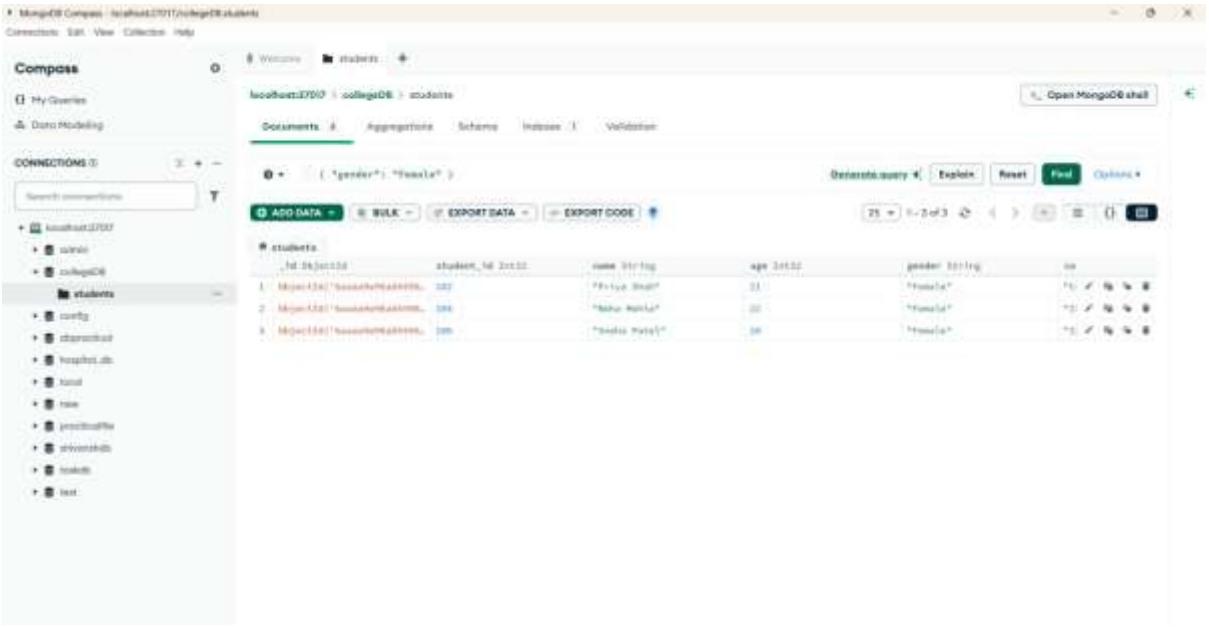


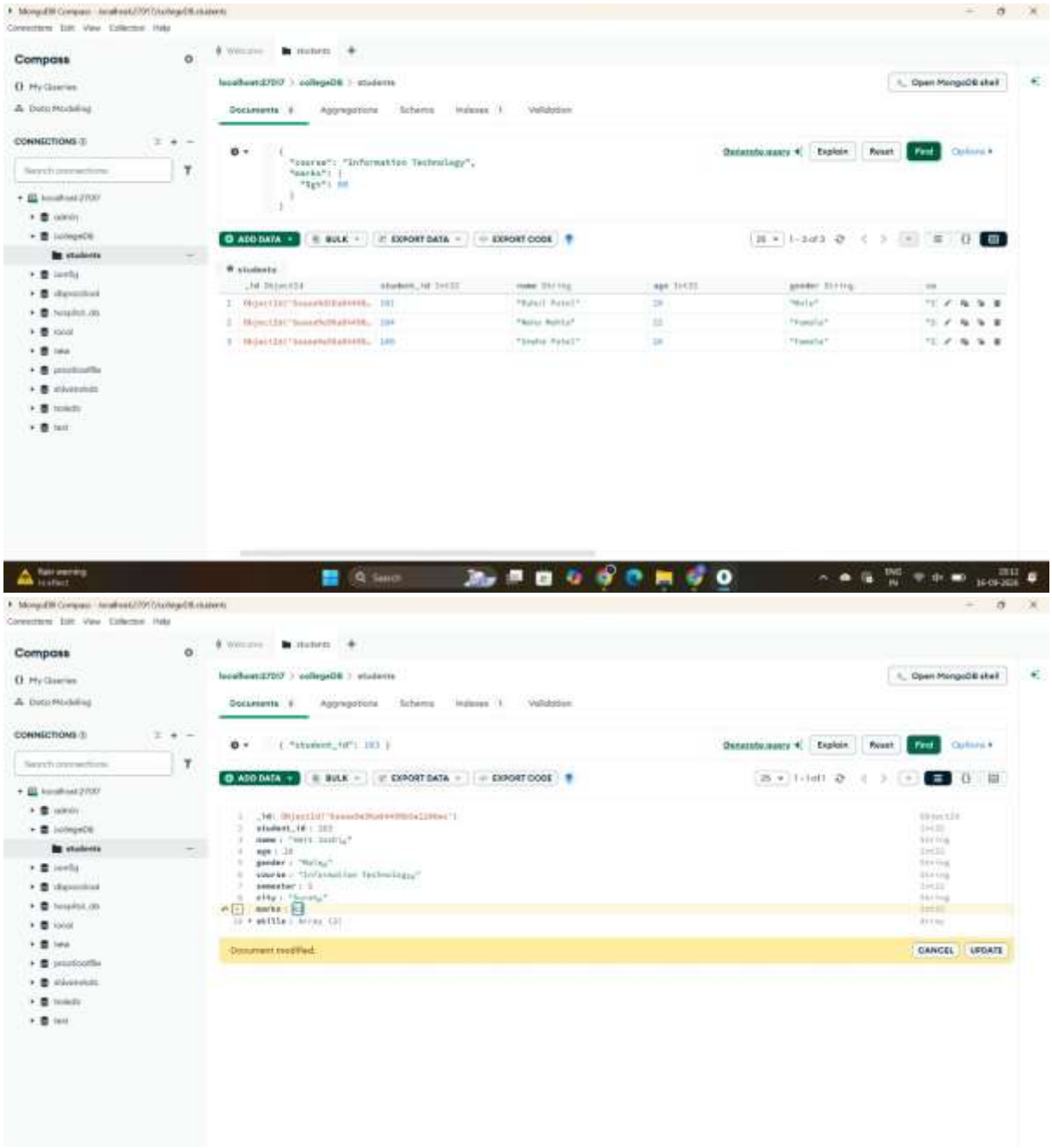


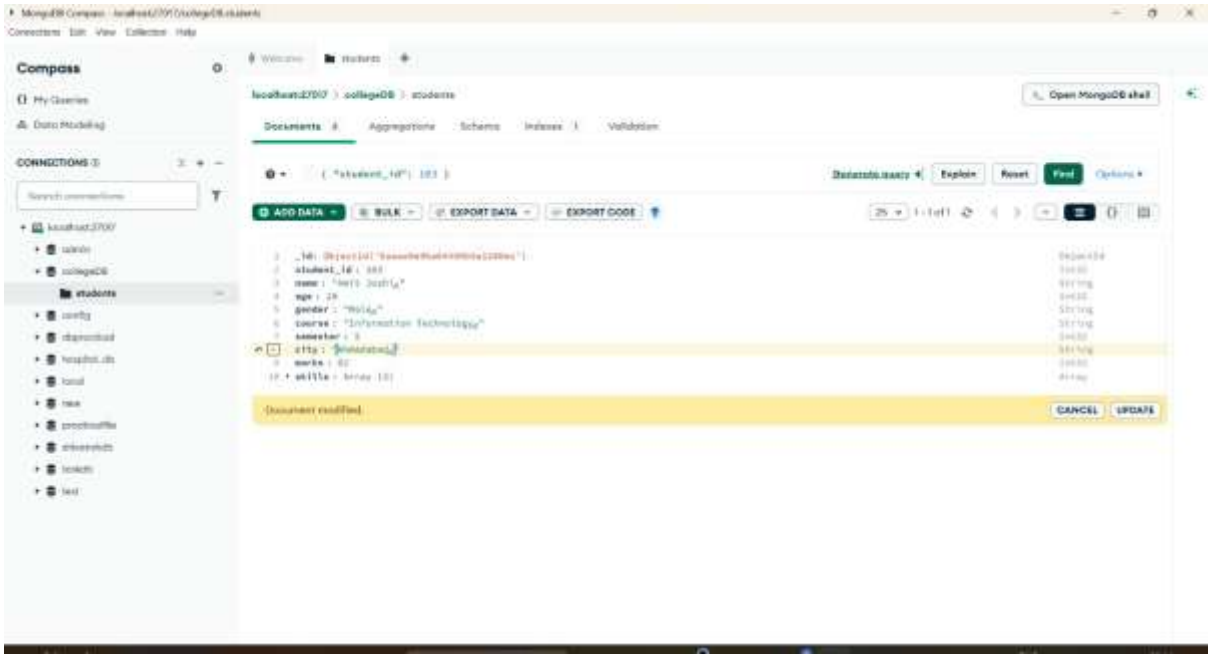


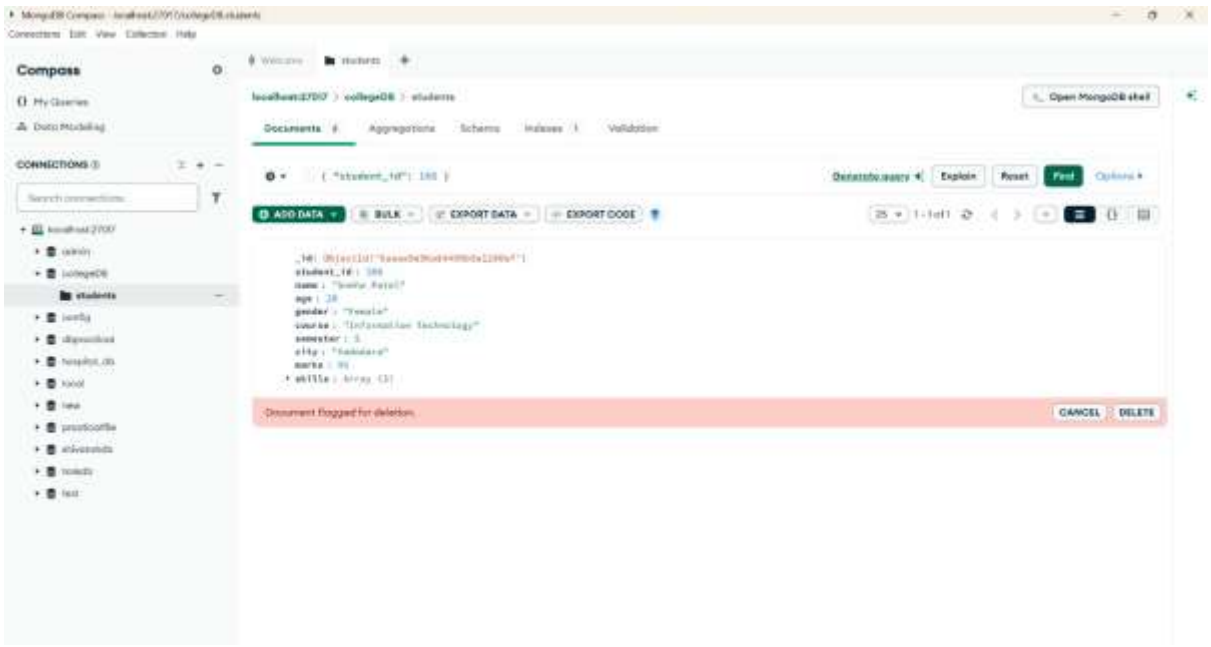


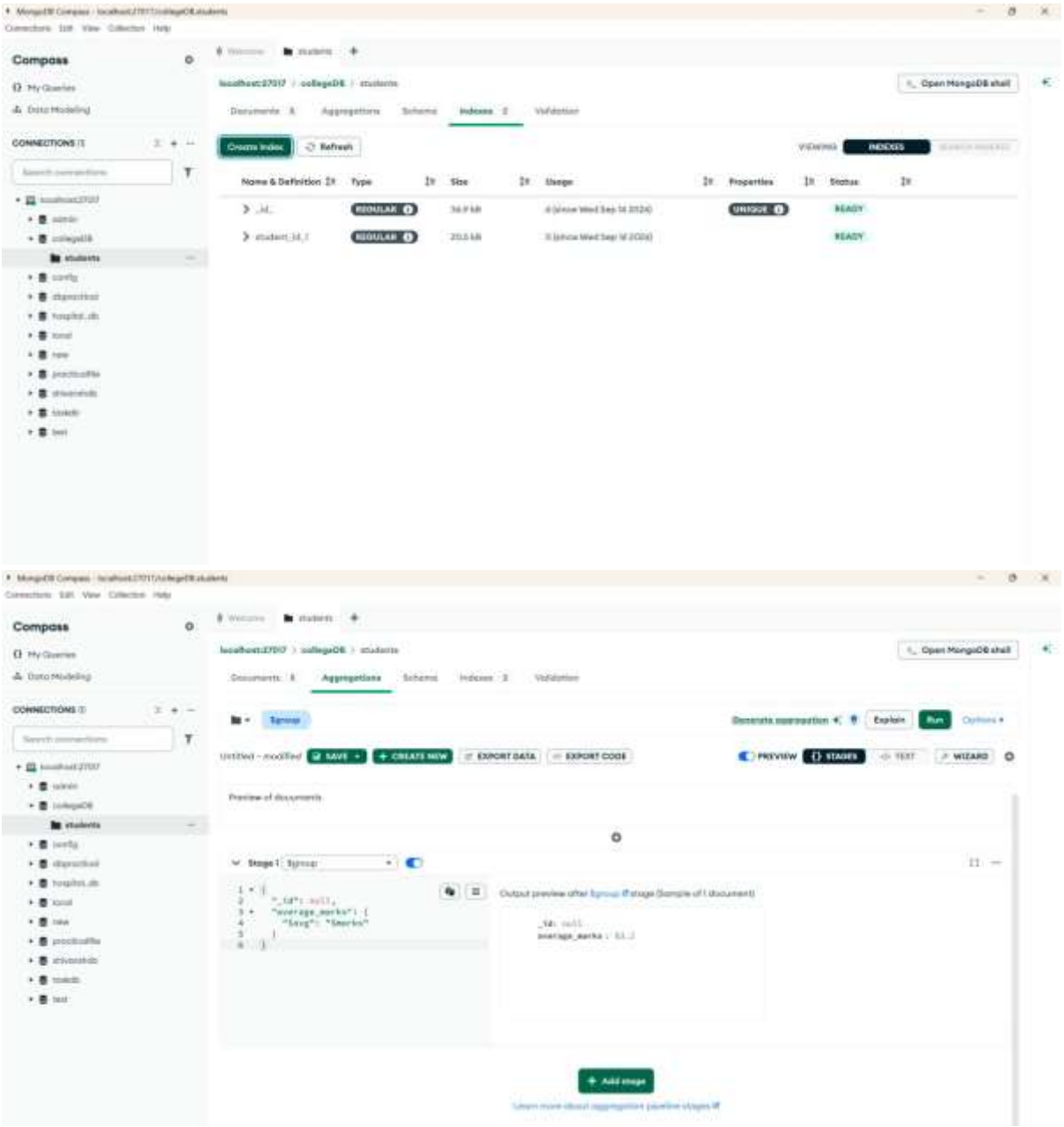

Compass
My Queries
Data Modeling
CONNECTIONS
localhost:27017
admin
collegeDB
students
config
objectsdbd
hospital_db
local
new
practicalfile
shivanishdb
tcetdb
test
localhost:27017 > collegeDB > students
Open MongoDB shell
Documents
Aggregations
Schema
Indexes
Validation
Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
students


Compass
My Queries
Data Modeling
CONNECTIONS
localhost:27017
admin
collegeDB
students
config
objectsdbd
hospital_db
local
new
practicalfile
shivanishdb
tcetdb
test
localhost:27017 > collegeDB > students
Open MongoDB shell
Documents
Aggregations
Schema
Indexes
Validation
{ "gender": "Female" }
Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
students

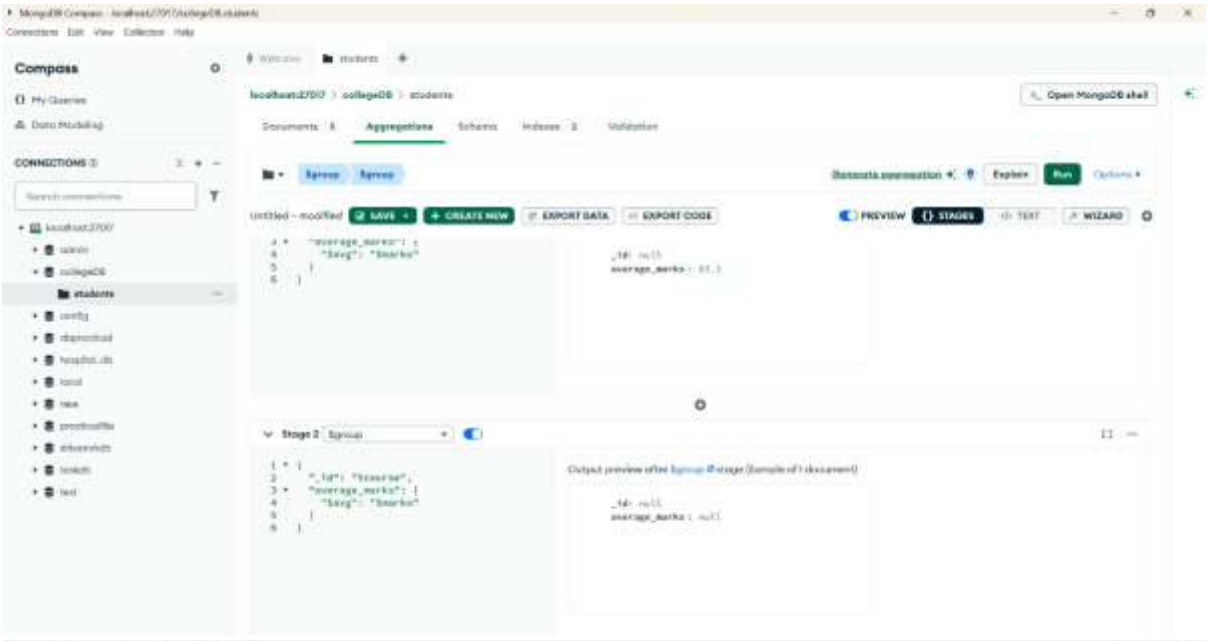


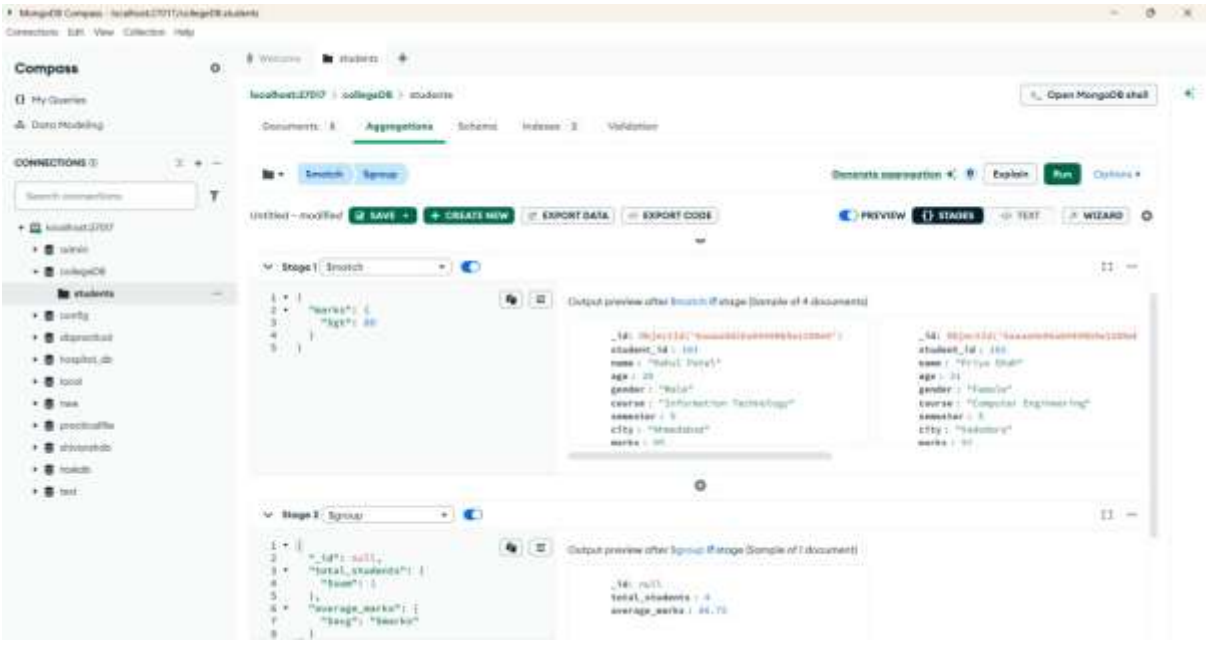

Compass
CONNECTIONS
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
Explain
Reset
Find
CANCEL
UPDATE

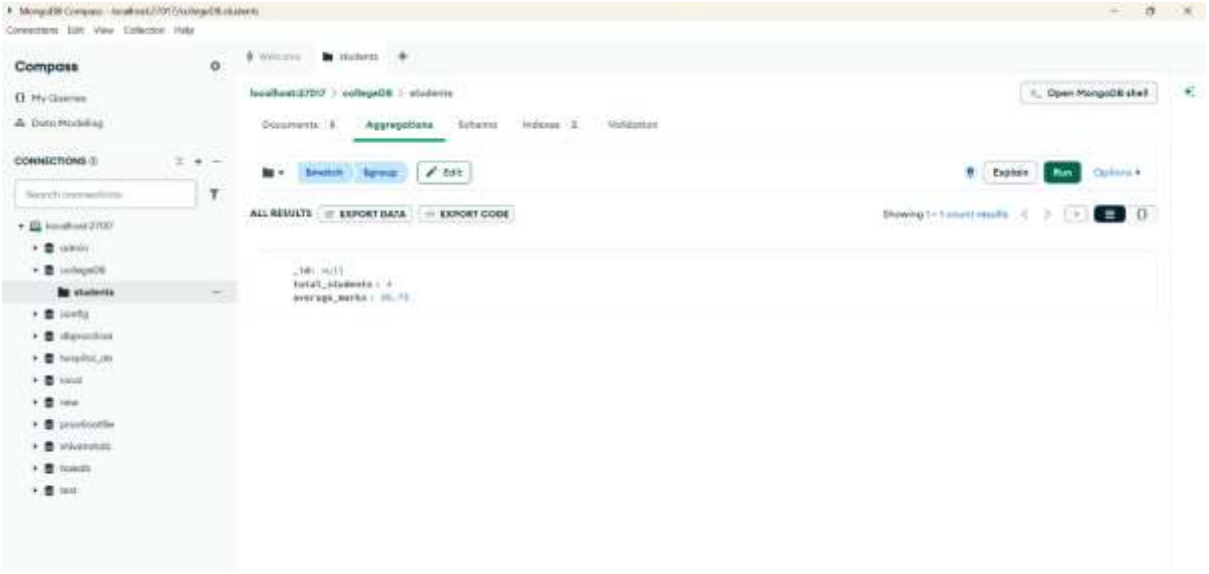

Compass
CONNECTIONS
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
Explain
Reset
Find
CANCEL
DELETE

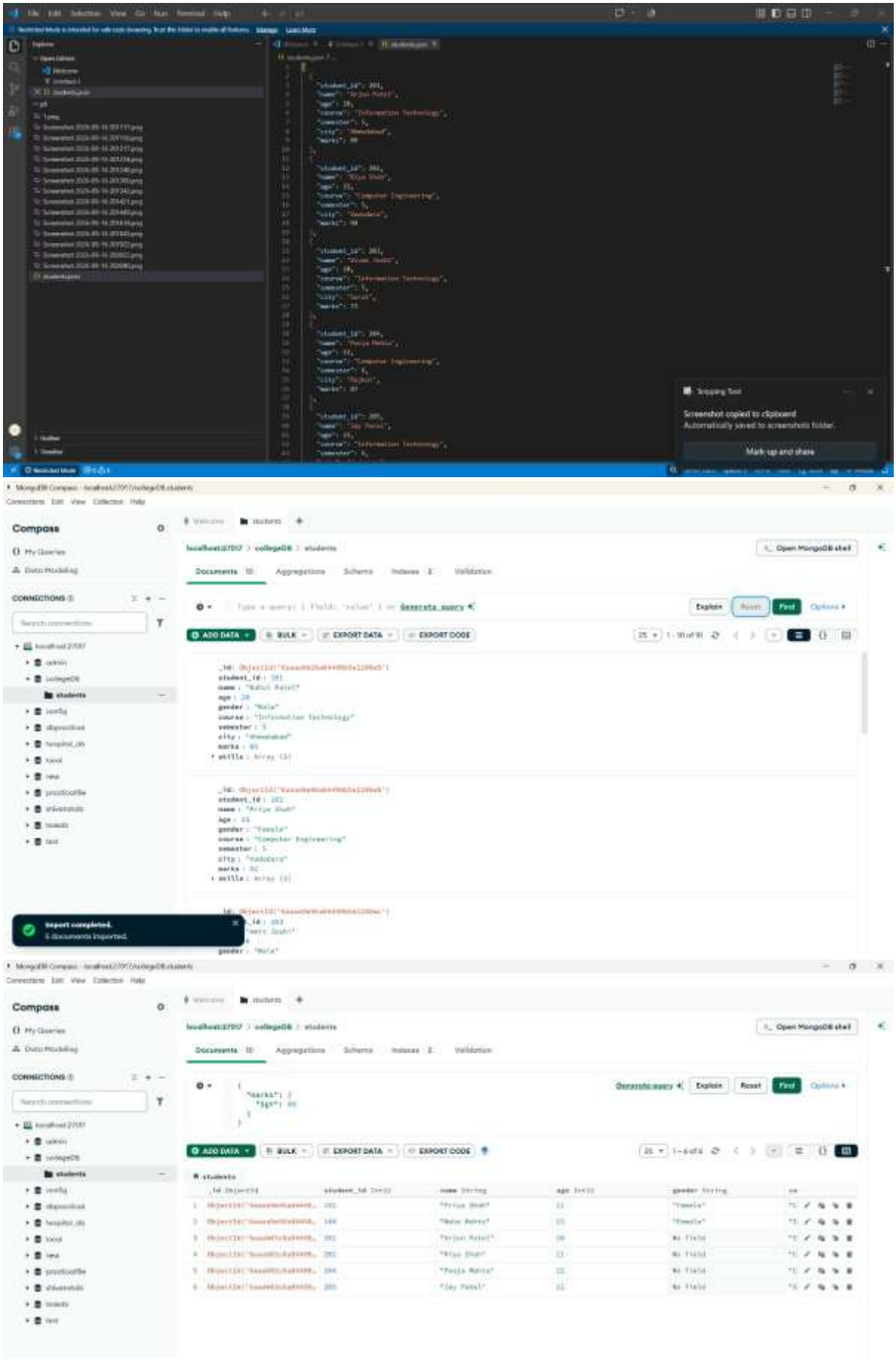

Compass
My Queries
Data Modeling
CONNECTIONS
Open MongoDB shell
Documents
Aggregations
Schema
Indexes
Validation
Create Index
Refresh
INDEXES
Name & Definition
Type
Size
Usage
Properties
Status
REGULAR
UNIQUE
READY

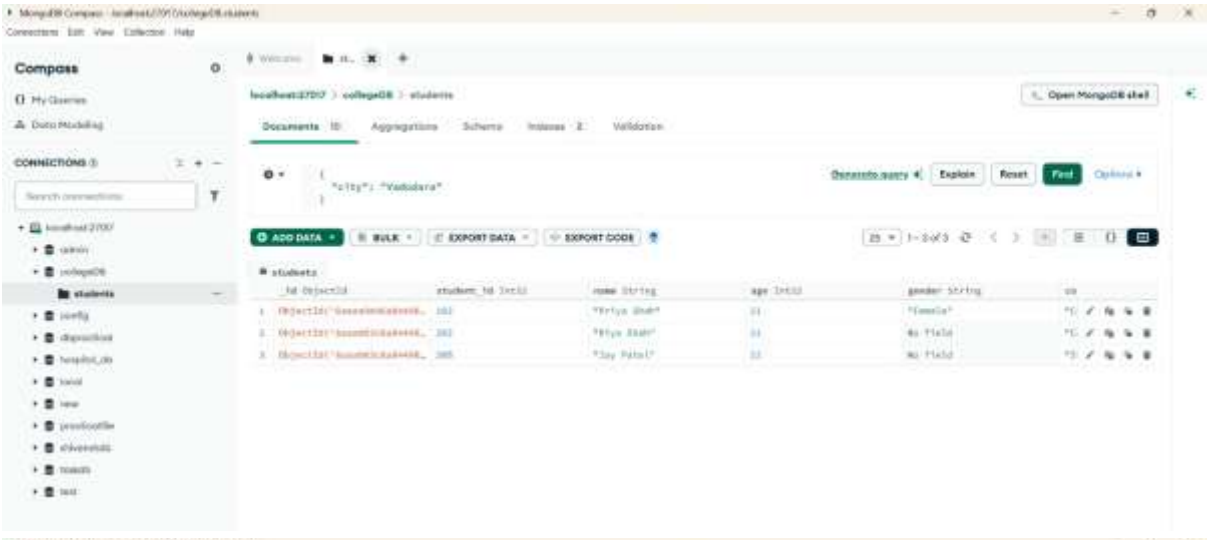
Compass
My Queries
Data Modeling
CONNECTIONS
Open MongoDB shell
Documents
Aggregations
Schema
Indexes
Validation
$group
Explain
Run
SAVE
CREATE NEW
EXPORT DATA
EXPORT CODE
PREVIEW
STAGES
WIZARD
Preview of documents
Stage 1
$group
Add stage

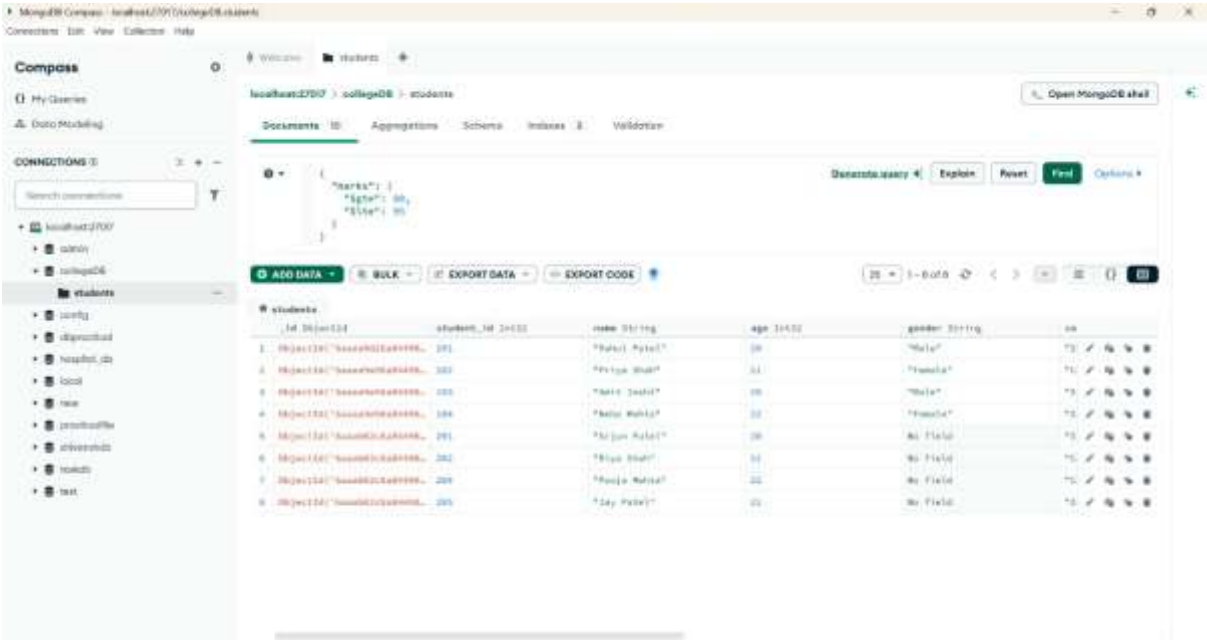







Compass
Import completed.
6 documents imported.
Compass





Conclusion

The MongoDB CRUD Operations practical was successfully completed using MongoDB Compass. A database and collection were created, and student documents were inserted and managed using Create, Read, Update, and Delete operations.

Different filtering and query operations were performed to retrieve required records. An index was created to improve query performance, and the Explain Plan was used to analyze query execution. The MongoDB aggregation pipeline was also used for analytical operations such as calculating average marks, grouping students, sorting records, and finding the top students.

The practical provided a better understanding of MongoDB, NoSQL document databases, BSON, CRUD operations, indexing, query processing, JSON data, and aggregation pipelines. These concepts are useful for managing flexible and large-scale data in Big Data applications.